🧠 Generic Agent Builder System — Detailed Design Document

---

1. 🎯 Project Overview (What are we building?)

We are building a **Generic Agent Builder System** that allows users to:

- Create AI agents using **natural language prompts**
- View and edit agent tools using a **UI**
- Improve tools using **remarks (feedback in plain English)**
- Run agents dynamically
- Use a **main orchestrator agent** to control everything

💡 Simple Analogy

Think of this like:

> 🧑‍💻 **VS Code + ChatGPT + Plugins → for AI agents**

- Agents = apps
- Tools = functions/plugins
- JSON = editable settings
- Orchestrator = operating system

---

2. 🏗️ System Architecture (Big Picture)

User → UI → Orchestrator Agent → System Tools → File System
↓
Agents + Tools

📁 Folder Structure (Final)

```
project/
│
├── agents/
│   ├── research_agent/
│   │   ├── agent.py            # runs the agent
│   │   ├── config.json         # editable config
│   │   ├── metadata.json       # optional info
│   │   └── tools/
│   │       ├── search_tool.py
│   │       ├── summarize_tool.py
│
├── core/
│   ├── agent_builder.py
│   ├── agent_loader.py
│   ├── patch_engine.py
│
├── orchestrator/
│   └── orchestrator_agent.py
│
├── generator/
│   └── agent_generator.py
│
├── ui/
│   └── app.py
```

---

3. ⚙️ Core Design Principles

✅ 1. Separation of Concerns

Layer	Purpose
Python (`.py`)	Executes logic

JSON (`config.json`)	Editable by user
UI	Displays + edits
LLM	Generates + refines

✅ 2. Controlled Editing

- Users **never edit Python directly**
- All edits happen through **JSON**
- LLM only generates **patches**, not full rewrites

✅ 3. Modular Agents

Each agent:

- has its own folder
- has its own tools
- is independently runnable

---

4. 📦 Agent Structure (Detailed)

🔹 Example: `config.json`

```
{
  "llm": {
    "model": "gpt-4",
    "temperature": 0
  },
  "tools": [
    {
      "id": "summarize",
      "file": "summarize_tool",
```

```
    "enabled": true,
    "config": {
      "prompt": "Summarize this:\n{input}"
    }
  },
  {
    "id": "search",
    "file": "search_tool",
    "enabled": true,
    "config": {}
  }
 ]
}
```

🧠 Explanation

Field	Meaning
`id`	unique identifier
`file`	Python file name
`enabled`	turn tool ON/OFF
`config`	editable parameters

---

5. 🧰 Tool Design (VERY IMPORTANT)

🔹 Example: `summarize_tool.py`

```
def get_tool():
  return {
    "name": "summarize",
    "description": "Summarizes text",
    "run": run
```

```
    }

def run(input_text, llm, config):
    prompt = config.get("prompt", "Summarize this:\n{input}")
    return llm.predict(prompt.format(input=input_text))
```

🧠 Key Rule

All tools MUST follow:

run(input_text, llm, config)

🔹 Example: `search_tool.py`

```
def get_tool():
    return {
        "name": "search",
        "description": "Search the web",
        "run": run
    }

def run(input_text, llm, config):
    return f"Searching for: {input_text}"
```

---

6. ⚙️ Agent Execution (Step-by-Step)

🔄 Flow

```
Load config.json
  ↓
Import tools dynamically
  ↓
Inject config into tool
  ↓
```

Create LangChain agent
↓
Run user query

🔹 Example

User asks:

"Summarize this article"

Agent:

1. Picks summarize tool
2. Applies prompt from config
3. Calls LLM
4. Returns result

---

7. 🖥️ UI Design (Detailed)

🔹 Top Bar

[Tools] [LLM] [Memory]

🔹 Tool Panel

Agent: research_agent

Tools:
[summarize] ✅
[search] ✅

🔹 Tool Editor

Tool: summarize

Prompt:
[Summarize this:\n{input}]

💬 Remarks:
[make it more detailed and use bullet points]

[Preview] [Apply]

---

8. 🔄 Remarks Feature (Core Innovation)

🎯 Goal

User should not edit config manually.
Instead:

> User speaks → system updates config

🔹 Example Flow

Current config:

"prompt": "Summarize this:\n{input}"

User says:

Make it more detailed and structured in bullet points

LLM returns:

{

```
  "operation": "update",
  "path": "config.prompt",
  "value": "Provide a detailed bullet-point summary:\n{input}"
}
```

Result:

"prompt": "Provide a detailed bullet-point summary:\n{input}"

---

9. ⚙️ Patch Engine (How updates work)

🔹 Code

```
def apply_patch(agent_config, tool_id, patch):
    for tool in agent_config["tools"]:
        if tool["id"] == tool_id:

            keys = patch["path"].split(".")
            ref = tool

            for k in keys[:-1]:
                ref = ref.setdefault(k, {})

            ref[keys[-1]] = patch["value"]

    return agent_config
```

🧠 What this does

- Finds the correct tool
- Navigates JSON path
- Updates only that field

---

10. 🤖 Orchestrator Agent (Brain of system)

🎯 Role

The orchestrator:

- understands user intent
- decides what action to take
- calls appropriate system tool

🔹 Example Tools

Tool	Purpose
create_agent	create new agent
edit_tool	update tool config
apply_patch	save changes
run_agent	execute agent

🔹 Example Interaction

User:

Create a research agent

→ Orchestrator calls:

create_agent

User:

Make summarize tool better

→ Orchestrator calls:

edit_tool

User:

Run the agent

→ Orchestrator calls:

run_agent

---

11. 🔧 Agent Generator

🎯 Input

"Create an agent that searches and summarizes content"

🎯 Output

Creates:

```
agents/research_agent/
  ├── agent.py
  ├── config.json
  ├── tools/
```

🔹 Generated config example

{

```
  "tools": [
    { "id": "search", "file": "search_tool", "enabled": true },
    { "id": "summarize", "file": "summarize_tool", "enabled": true }
  ]
}
```

---

12. 🔒 Constraints (IMPORTANT)

✅ Allowed

- Edit prompts
- Enable/disable tools
- Modify config fields

❌ Not Allowed

- Editing Python via LLM
- Arbitrary code execution
- Breaking tool interface

---

13. 🚀 Future Enhancements

🔹 Tool Testing Panel

Run tool before saving

🔹 Versioning

v1 → v2 → v3

🔹 Tool Marketplace

Reusable tools

🔹 Multi-Agent System

Agents talking to each other

---

14. 🧠 Final System Flow

User Prompt
↓
Agent Generated
↓
User edits tool (remarks)
↓
LLM → JSON Patch
↓
Preview → Apply
↓
Save config
↓
Orchestrator runs agent

✅ 15. Summary

This system is:

- 🧠 Agent Builder
- 🛠️ Tool Editor
- 🤖 Orchestrator System

🔥 Key Strengths

- Natural language control
- Safe editing
- Modular design
- Scalable architecture

🧠 Final Insight

You’re not just building:

> “an agent generator”

You are building:

> **A full Agent Development Environment (Agent IDE)**

End of Document